Chapter 5

The Technological Framework and Software Stack

5.1 Backend Infrastructure and Data Platform Architecture

To operationalize the domain-oriented Data Product model and support interactive analytical workloads within Amadori S.p.A., the enterprise data architecture is undergoing an evolutionary transition from an on-premises relational infrastructure toward a scalable, cloud-native data platform.

The baseline enterprise environment relies primarily on an on-premises Oracle database, which historically serves as the operational transaction repository and the target for analytical queries. To overcome the computational constraints of centralized on-premises storage and eliminate contention between transactional processing and resource-intensive analytical calculations, the target architecture introduces Google BigQuery as the enterprise cloud data warehouse. BigQuery provides a serverless, highly scalable environment based on distributed compute engines and columnar storage, enabling parallelized execution of complex analytical queries over large-scale historical datasets.

Rather than implementing an abrupt cutover, the transition follows a phased migration strategy. In the initial phase, analytical assets and ingestion pipelines continue to read directly from the operational Oracle database. As data models, historical partitions, and ETL routines are progressively refactored and validated, tables are transferred to Google BigQuery. During this transitional period, both database connections remain active simultaneously.

The presence of a centralized Semantic Layer implemented in Looker plays a strategic role in facilitating this dual-connection phase. Because Looker functions via in-database querying rather than data extraction, it can maintain concurrent

dialect connections to both Oracle and BigQuery. Business dimensions and metrics formalized in LookML can initially target tables hosted on the Oracle instance; subsequently, as individual domains complete their migration to BigQuery, the underlying connection references within the LookML models are updated to point to the cloud tables. This decoupling ensures that downstream analytical consumers, executive reports, and AI interfaces remain completely shielded from the physical migration process, experiencing no disruption in metric definitions or query behavior while backend storage is shifted.

5.2 Formalizing Business Data: The Semantic Layer with Looker and LookML

To operationalize the conceptual definitions established by the enterprise ontology and provide an auditable single source of truth across analytical processes, the architecture implements Looker and its declarative modeling language, LookML. In this stack, Looker does not operate merely as a data visualization layer, but acts as the centralized execution engine for the enterprise Semantic Layer [21]. Looker operates on an in-database query paradigm: analytical data remains stored within the underlying database, while the semantic platform dynamically compiles user interactions, dashboard requests, and API calls into optimized SQL queries executed directly by the target database engine at runtime [23].

This in-database approach plays a crucial role during the transition from the on-premises Oracle database toward Google BigQuery. Currently, the operational material planning pipeline populates the core fact table (`F_MATERIAL_PLANNING`) via a daily scheduled batch job running directly on the Oracle database. While the long-term target architecture aims to migrate these workloads to Google BigQuery, the abstraction provided by LookML allows the organization to manage this shift transparently. Because database connections are configured at the model level, LookML isolates physical connection dialects and schema differences. When tables are migrated to BigQuery in future deployment phases, connection strings and dialect syntax are updated within the model files without altering the exposed business entities, ensuring zero downtime and complete consistency for downstream users and cognitive agents.

5.2.1 Architectural Best Practices and Performance Optimization

Designing a high-throughput Semantic Layer requires balancing analytical flexibility with computational efficiency. When modeling complex supply chain dynamics

in LookML, several engineering best practices were applied to minimize query execution overhead and prevent maintenance bottlenecks [11]:

- **Pushing transformations to the backend:** Complex transformations, recursive allocations, and historical rolling aggregations are executed upstream during database batch processing rather than calculated dynamically inside LookML. Looker is dedicated to aggregation, dimensional filtering, and metric definition, preventing resource-intensive row-level computations at query runtime.

- **Denormalization of frequently filtered attributes:** While relational normalization minimizes redundancy in transactional platforms (OLTP), analytical queries run faster when frequently joined descriptive attributes are pre-aggregated or denormalized into wide structures, reducing join depth.

- **Handling surrogate keys and natural codes:** Redundant internal surrogate keys are systematically hidden from the interface. Instead, the model exposes standardized, concatenated business attributes (combining natural alphanumeric codes with their descriptions), streamlining both human analysis and machine interpretation.

- **Minimizing runtime join overhead in dashboard visualizations:** While an Explore model file defines an extensive relational network to grant complete self-service flexibility, individual dashboard tiles must query only the strictly necessary views. Looker generates SQL dynamically based on the dimensions selected in each tile; keeping dashboard queries isolated to minimal join paths directly reduces runtime join operations on the database engine, eliminating computational bottlenecks and query execution delays.

- **Curating the semantic exposure boundary:** Fields that are not explicitly declared in LookML remain completely invisible to consuming tools. Restricting exposed dimensions to certified attributes establishes an architectural boundary that protects users from deprecated columns.

5.2.2 Centralized Governance and the DRY Principle in Security Design

A foundational engineering objective of formalizing business semantics in LookML is strict compliance with the *Don't Repeat Yourself* (DRY) principle, which directly prevents code duplication and long-term maintenance overhead across enterprise analytics.

In conventional reporting architectures, enforcing data access segregation between distinct operational entities typically generates architectural anti-patterns. Technical teams are frequently forced to replicate entire models, SQL queries, or dashboard copies simply to hardcode facility-specific filtering criteria (e.g., maintaining cloned reports for each regional logistics deposit) or to strip out sensitive data columns for non-executive audiences. This duplication incurs severe operational risks: whenever an underlying business logic changes—such as updating the calculation formula of gross material requirements or shifting the baseline demand threshold—the alteration must be manually propagated across every replicated asset, creating high maintenance costs, security vulnerabilities, and eventual metric drift.

Within the LookML framework, operational business definitions and data security boundaries are decoupled, enabling the enterprise to enforce granular data access over a single, unified codebase:

- **Dynamic Row-Level Security (RLS) via `access_filter`:** Rather than creating dedicated Explores or dashboards for different facilities, row-level boundaries are enforced dynamically at query generation time. Through Looker's `access_filter` parameter, the platform maps an authenticated user's session attribute (e.g., `piattaforma_autorizzata`) directly to the physical platform column (`l_ptf.piattaforma_cod`). Because Looker natively supports multi-value user attributes and wildcards, this mechanism accommodates both local logistics operators restricted to a specific set of platforms and central planning teams authorized to inspect the entire operational network. When a user runs a query or loads a dashboard, Looker dynamically injects the parameterized filtering predicate into the SQL `WHERE` clause, ensuring strict data compartmentalization across the multi-site supply chain without duplicating a single line of modeling code.

- **Granular Column-Level Security (CLS) and Access Grants:** Field visibility is governed systematically at the semantic declaration level rather than by partitioning underlying database tables. Non-actionable technical fields—such as surrogate identifiers, system timestamps, and internal status flags—are universally obscured from user visibility using `hidden: yes`, preventing clutter in the Explore interface and filtering unnecessary noise from the AI agent's semantic context. Furthermore, to safeguard confidential commercial or financial metrics without generating separate reporting views, LookML provides `required_access_grants`. For instance, while gross material volumes (`fabbisogno_materiale`) are accessible to all operational planners, a related measure tracking standard component unit costs (`costo_standard_materiale`) can be locked behind an access grant tied to an executive or managerial permission set. Users or agents lacking that

explicit entitlement are completely prevented from querying the restricted measure or seeing it in their semantic schema, ensuring security by design while maintaining a single, centralized model.

5.2.3 Implementation: Material Planning Fact and Master Views

The operationalization of the Material Planning engine described in Chapter 3 is implemented through a set of dedicated LookML views and explores. The primary fact view, `f_material_planning`, defines the daily gross material requirements across forecast dates, finished items, logistics platforms, and packaging or ingredient components:

Listing 5.1: LookML Definition for Material Planning Fact View

```
view: f_material_planning {
  sql_table_name: `gsc-dp-mt-plan-sbx.
     material_planning_export.F_MATERIAL_PLANNING` ;;
  label: "Fabbisogno lordo materiali"

  dimension: primary_key {
    primary_key: yes
    type: string
    sql: concat(cast(${data_mp} as string), '-', cast(${
       art_id} as string), '-',
                cast(${ptf_id} as string), '-', cast(${
                   mat_id} as string), '-',
                cast(${mat_um_id} as string)) ;;
    description: "Chiave primaria univoca: DATA_MP x
       ART_ID x PTF_ID x MAT_ID x MAT_UM_ID"
    hidden: yes
  }

  dimension_group: data_previsione {
    type: time
    label: "Data Previsione"
    timeframes: [raw, date, week, month, quarter, year]
    datatype: date
    convert_tz: no
    sql: parse_date('%Y%m%d', cast(${data_mp} as string)
       ) ;;
```

```
    description: "Data di previsione del fabbisogno
      lordo"
    synonyms: ["data di previsione", "data fabbisogno",
      "orizzonte temporale", "data pianificata"]
  }

  measure: fabbisogno_materiale {
    type: sum
    label: "Fabbisogno Materiale"
    sql: ${mat_qta} ;;
    description: "Fabbisogno lordo del materiale (
      imballo, etichetta o ingrediente) necessario a
      coprire la produzione pianificata, per deposito,
      data e unita di misura"
    value_format_name: decimal_0
    synonyms: ["fabbisogno lordo", "quantita materiale",
       "consumo previsto", "fabbisogno totale"]
    drill_fields: [dettaglio_fabbisogno*]
  }

  set: dettaglio_fabbisogno {
    fields: [materiale.materiale, materiale.
      tipo_materiale, data_previsione_date,
      fabbisogno_materiale, um_cod]
  }
}
```

In this view, technical surrogate keys (`art_id`, `mat_id`, `ptf_id`) and raw numeric dates are suppressed from user visibility using `hidden: yes`. Instead, the view exposes a clean `dimension_group` parsing the integer date into standard calendar intervals. Business measures—such as `fabbisogno_materiale`—incorporate pre-configured `drill_fields`, enabling users to click on any aggregated volume to immediately inspect the underlying constituent articles and material classifications.

Master data views follow the same design. In `l_art_jde`, physical columns are enriched to dynamically distinguish between parent finished items, neutral items (*PN*), and material components:

Listing 5.2: LookML Master Data View with Derived Categorization

```
view: l_art_jde {
  sql_table_name: `gsc-dp-mt-plan-sbx.
```

```
    material_planning_export.L_ART_JDE` ;;
  view_label: "Articolo"

  dimension: articolo {
    type: string
    label: "Articolo"
    sql: concat(${TABLE}.ART_JDE_LCOD, ' - ',${TABLE}.
       ART_JDE_DES) ;;
    description: "Codice e descrizione alfanumerica dell
       'articolo finito"
    synonyms: ["prodotto finito", "articolo di vendita",
        "prodotto", "PF", "PFL", "PFN"]
  }

  dimension: tipo_materiale {
    type: string
    label: "Tipo Materiale"
    sql: case
           when ${TABLE}.ART_JDE_LCOD like 'I%' then '
              Imballo'
           when ${TABLE}.ART_JDE_LCOD like '0%' then '
              Ingrediente'
           else 'Altro'
         end ;;
    description: "Classificazione del materiale tra
       imballaggio e ingrediente primario"
    synonyms: ["categoria materiale", "tipologia
       componente", "imballo vs ingrediente"]
    suggestions: ["Imballo", "Ingrediente"]
  }
}
```

5.2.4 Model Definition: Relational Joins and Access Perimeter

The relational orchestration of the Material Planning domain is codified within the LookML model file (`material_planning.model.lkml`). The model specifies the active database connection, imports the view definitions, and defines the join topology centered around the primary fact view:

Listing 5.3: LookML Model and Explore Configuration for Material Planning

```
connection: "materialplanningbq"

include: "/views/*.view.lkml"

explore: f_material_planning {
  label: "Fabbisogno lordo materiali"
  sql_always_where: 1=1 ;;

  # access_filter: {
  #   field: l_ptf.piattaforma_cod
  #   user_attribute: piattaforma_autorizzata
  # }

  join: articolo {
    from: l_art_jde
    view_label: "Articolo"
    type: inner
    relationship: many_to_one
    sql_on: ${f_material_planning.art_id} = ${articolo.
       art_jde_id} ;;
    fields: [articolo.articolo, articolo.articolo_attivo
       , articolo.articolo_neutro_cod]
  }

  join: materiale {
    from: l_art_jde
    view_label: "Materiale"
    type: inner
    relationship: many_to_one
    sql_on: ${f_material_planning.mat_id} = ${materiale.
       art_jde_id} ;;
    fields: [materiale.materiale, materiale.
       tipo_materiale]
  }

  join: l_ptf {
    type: inner
    relationship: many_to_one
    sql_on: ${f_material_planning.ptf_id} = ${l_ptf.
```

```
      ptf_id} ;;
  }

  join: l_um {
    type: inner
    relationship: many_to_one
    sql_on: ${f_material_planning.mat_um_id} = ${l_um.
      um_id} ;;
  }

  join: l_lin {
    type: inner
    relationship: many_to_one
    sql_on: ${articolo.art_jde_lin_id} = ${l_lin.lin_id}
      ;;
  }

  join: l_fam {
    type: inner
    relationship: many_to_one
    sql_on: ${articolo.art_jde_fam_id} = ${l_fam.fam_id}
      ;;
  }
}
```

As demonstrated in Listing 5.3, the Explore implements a role-playing dimension by joining the master table `l_art_jde` twice under two distinct semantic contexts: once as `articolo` (representing the finished commercial product generating demand) and once as `materiale` (representing the physical packaging or ingredient consumed). Furthermore, by explicitly filtering exposed attributes via the `fields` parameter, the model guarantees that only certified dimensions reach the analytical and conversational interfaces.

5.2.5 Semantic Modeling for Self-Service Exploration and AI Enablement

The enrichment of LookML parameters serves two interdependent consumption modes: human self-service analytics and cognitive AI agents.

For corporate planners, the centralized Explore defines certified relational joins across logistics platforms (`l_ptf`), units of measure (`l_um`), production lines (`l_lin`), and the dual aliases of `l_art_jde` (mapped once as the parent *Articolo* and once as

the dependent *Materiale*). This semantic foundation enables intuitive self-service BI: business users can build ad-hoc operational queries in the Explore interface, save customized views (*Looks*), and combine them into interactive, cross-filtered dashboards without writing SQL code.

Simultaneously, this enriched metadata provides the deterministic guardrails required for autonomous Generative AI agents [10]:

- **Vocabulary Disambiguation via Synonyms:** Natural language queries rarely match exact database column names. The `synonyms` parameter maps common enterprise jargon, informal acronyms (e.g., "PFL", "PFN", "deposito"), and domain terms directly to validated LookML dimensions, allowing an AI agent to match user intent with deterministic schema fields.

- **Semantic Anchoring via Descriptions:** The `description` parameter defines the exact operational scope and mathematical logic of each metric. This metadata prevents language models from misinterpreting raw values or fabricating fictitious aggregation logic.

- **Context Pruning via Hidden Fields:** By hiding raw IDs and technical join keys (`hidden: yes`), the Semantic Layer filters out noise from the agent's context window. The LLM evaluates only certified, business-relevant fields, substantially reducing prompt token consumption and preventing hallucinated joins [10].

5.3 Governance Roadmap: Extending Lineage and Enterprise Knowledge via OpenMetadata

While Looker operates as the centralized semantic compiler and enforces metric governance at the analytical layer, comprehensive visibility across the broader enterprise data lifecycle requires a dedicated metadata management platform. Currently, business definitions, dimensional attributes, and operational descriptions are codified and maintained directly within LookML files using parameters such as `description`, `label`, and `synonyms`. While this approach successfully establishes LookML as an executable "Business Glossary as Code", it remains primarily accessible to analytics engineers and technical practitioners working within the version-controlled modeling environment.

To decouple business glossary governance from technical codebases and achieve automated data lineage across heterogeneous systems, the enterprise architecture envisions the integration of OpenMetadata as an evolutionary roadmap milestone.

OpenMetadata is an open-source, schema-first metadata management platform built upon standard JSON schemas for data discovery, governance, and quality profiling. Within the planned roadmap, OpenMetadata will expand enterprise governance beyond the analytical consumption boundary, focusing on three foundational capabilities:

1. **Centralized, Low-Code Business Glossary:** Transitioning business definitions from plain-text LookML parameters into an intuitive enterprise catalog accessible to non-technical business stakeholders [5]. This provides domain stewards across planning, supply chain, and commercial divisions with the tooling required to manage term ownership, certify metric definitions, and approve terminology changes collaboratively without committing code to a Git repository.

2. **Automated End-to-End Data Lineage:** Establishing systemic traceability across the entire data lifecycle. By connecting directly to source catalogs—tracing tables from their operational origin in the on-premises Oracle database, through the cloud landing zones and transformations in Google BigQuery, down to the views, dimensions, and measures compiled by Looker. Automated lineage tracking will enable data engineering teams to perform proactive impact analysis, accurately identifying which downstream Explores, operational dashboards, or predictive material planning routines will be disrupted prior to executing any upstream schema migration.

3. **Enterprise Knowledge Graph Construction:** Structuring enterprise metadata entities—including relational storage assets, batch pipeline schedules, semantic metric definitions, and user access policies—as an interconnected knowledge graph [15]. This structural context provides deterministic insights into data provenance and reliability, serving as an authoritative grounding foundation to enhance future autonomous AI workflows.

In this target framework, OpenMetadata will ingest metadata programmatically from Looker and underlying database instances via dedicated metadata APIs, eliminating manual synchronization overhead. This evolutionary enhancement strictly decouples operational query execution (handled in-database by Looker, Oracle, and BigQuery) from systemic metadata governance, completing the structural transition toward enterprise-grade Data Products.

5.4 Conversational AI Enablement: Gemini in Looker and Iterative Agent Tuning

To democratize analytical access for enterprise decision-makers and operational planners, the data platform integrates Google Cloud's native Generative AI capabilities through **Gemini in Looker**. Rather than querying physical database engines directly with unconstrained text-to-SQL generation—an approach notoriously vulnerable to hallucinated schema joins and non-existent business definitions—the conversational interface is strictly anchored to the certified Semantic Layer [8, 12].

5.4.1 Ecosystem Overview: Conversational Modalities in Looker

Google Cloud structures natural language analytics in Looker across three specialized conversational modalities:

- **Chat with Dashboard:** A conversational assistant anchored to a precomputed analytical dashboard. It operates exclusively on the rendered visualizations and pre-aggregated tiles, providing immediate qualitative summaries, identifying visual anomalies, and explaining trends without executing ad-hoc database queries.
- **Chat with Explore:** A contextual assistant embedded within a single Explore interface. It assists technical analysts in constructing ad-hoc visual queries by mapping natural language prompts to available dimensions, measures, and filters within that specific Explore boundary.
- **Conversational Data Agents (Chat with Your Data):** The enterprise architectural choice implemented in this project. Enabled via Looker Admin and the Conversational Analytics framework, this modality provisions an autonomous agent powered by Gemini that can bind up to five certified Explores [12]. For the Amadori material planning use case, the agent connects directly to the `f_material_planning` explore within the `material_planning` model, acting as an interactive operational assistant governed by custom domain instructions and LookML metadata.

5.4.2 Iterative Tuning Methodology: The "Show Thinking" Feedback Loop

In alignment with official engineering guidelines for Looker conversational analytics [12], configuring an enterprise-grade agent is not a static task, but an empirical,

iterative engineering workflow. As illustrated in **??**, the tuning process follows a closed feedback loop: executing domain-specific natural language queries, inspecting the model's step-by-step reasoning via the native **Show Thinking** diagnostic interface, identifying semantic misalignments, and mitigating them either upstream in the LookML model or downstream in the Agent's system instructions.

During validation against real Amadori operational scenarios, this inspection cycle revealed several critical behavioral challenges that were systematically resolved:

1. Lexical and Semantic Disambiguation When testing raw queries such as *"Di quante etichette avrò bisogno il prossimo mese?"*, the agent initially returned null results. Inspecting the *Show Thinking* log revealed that Gemini attempted an unguided string filter:

```
Filtered data for L_MAT.MAT_DES with value %ETICHETTA%, %Etichetta%,
%etichetta%...
```

In the corporate ERP, label descriptions are systematically abbreviated with prefixes such as `"ET"` or `"ETICH"`. Rather than relying on trial-and-error, the agent's instructions were enriched with domain syntax rules: *"Le etichette pianificate iniziano con 'Et' o includono 'Etich'"*. Similarly, colloquial terms were mapped to official database terms: requests for *"vaschette"* failed until an instruction established that *"vaschetta"* and *"vassoio"* are exact functional synonyms.

2. Entity Consolidation and Canonical Filtering When querying requirements for regional hubs (e.g., *D̈i cosa avrò bisogno a Santa Sofia fino a fine anno?¨*), the agent stalled, prompting the user for clarification because multiple operational codes map to that geographic hub. To prevent interactive bottlenecks, a deterministic system instruction was introduced: whenever an ambiguous reference to a plant or platform is detected, the agent automatically applies a canonical set filter on the platform code (`l_ptf.ptf_cod` in `[S01, S02, S04, S05, S10, S50]`).

3. Schema Refactoring: From Code Interpreter to Semantic Dimension A key turning point in the tuning process occurred during requests to separate ingredients from packaging: *"Dividi gli ingredienti dagli imballi"*. In the *Show Thinking* log, Gemini activated its Python-based Code Interpreter, engaging in multi-step heuristic reasoning:

> *"Codes starting with '08' seem to correspond to 'Budello' and 'Olio' (ingredients), while codes starting with 'I' or 'IM' correspond to 'Et termica',*

'Vassoio', 'Film' (packaging)... I will perform a query and use python_agent to categorize them..."

While the agent successfully inferred the correct classification, this runtime heuristic introduced severe response latency. Guided by the principle of pushing logic to the semantic core, this insight was leveraged to permanently enrich the LookML view: a dedicated dimension, `tipo_materiale`, was codified with a deterministic `CASE` statement (Listing 5.2). This architectural shift eliminated runtime LLM deliberation, transforming an ad-hoc statistical inference into an indexed, reproducible dimensional filter.

4. Code Disambiguation and the "Superset Trap" When prompted with an exact article code (e.g., *"Quanti ingredienti mi servono per l'articolo 00700?"*), the language model exhibited the "superset trap," hesitating and asking whether the user meant `00700`, `00700F`, `100200700`, or `100700001`. To enforce operational precision, an explicit constraint was codified: *"Se l'utente fornisce un codice, questo deve essere trattato come corrispondenza esatta (EXACT_MATCH), senza richiedere conferme su codici contenitori se il codice esiste nel database"*.

5. Computational Pruning and Temporal Fallback To avoid massive, unbounded SQL table scans when users omit temporal boundaries, a default fallback instruction was mandated: any query lacking an explicit timeframe automatically defaults to the interval spanning from the current execution date to the end of the subsequent calendar month. This bounds query costs on the database while delivering immediate operational relevance.

5.4.3 Architectural Constraints: Heterogeneous Units of Measure and Dashboard Routing

A fundamental limitation of LLM-driven conversational interfaces emerged when handling multi-unit aggregations:

$$\text{Total Gross Requirement} = \sum \text{Packaging (NR)} + \sum \text{Film (MT)} + \sum \text{Trays (MQ)} + \sum \text{Spices (} \tag{5.1}$$

In relational databases, summing numerical values across incompatible Units of Measure (UoM) produces a meaningless aggregate. Furthermore, when users instructed the agent to *"generate separate charts for each unit of measure"*, the agent encountered an architectural constraint: Looker Conversational Agents cannot perform runtime looping to dynamically generate N independent visualization widgets within a single conversational turn. Attempting to execute such requests resulted in a single SQL query being rendered identically across duplicated widgets.

To solve this semantic and visualization challenge, the architecture decouples exploratory conversational dialogue from multi-dimensional structured reporting:

1. **Certified LookML Dashboard Deployment:** A dedicated, governed dashboard (`fabbisogni_um`) was developed in LookML, featuring four decoupled column tiles, each strictly bound to a single unit of measure (`MT`, `MQ`, `NR`, `KG`) and controlled by global platform and date filters.

2. **Deterministic Agent Routing via Deep-Linking:** When an incoming user query requests an aggregate volume analysis spanning multiple UoMs, the agent is instructed not to generate a flat chart. Instead, it explains the mathematical necessity of keeping units segregated and dynamically constructs a parameterized, absolute URL pointing to the certified dashboard, pre-populating the URL query string with the user's specific context (e.g., `?filtro_citta=S01&filtro_periodo=2026-11`).

3. **Sanitization and Enterprise Security Handshake:** During testing, it was observed that Looker's markdown sanitizer suppressed relative dashboard hyperlinks generated by the LLM (rendering an unlinked anchor tag `<a>` to prevent arbitrary URL injection). To ensure seamless user navigation, the instruction was refined to mandate absolute URLs using the official instance domain, returning the link as plain text with clear instructions for the user to open it in an isolated browser session.

This hybrid design encapsulates the core philosophy of the proposed data architecture: leveraging Generative AI for intent recognition, lexical routing, and contextual guidance, while delegating complex multi-metric visualization to the deterministic, governed Semantic Layer.

Bibliography

[1] Christine L. Borgman. *Big Data, Little Data, No Data: Scholarship in the Networked World*. MIT Press, Cambridge, MA, 2015.

[2] Geoffrey C. Bowker and Susan Leigh Star. *Sorting Things Out: Classification and Its Consequences*. MIT Press, Cambridge, MA, 1999.

[3] George E. P. Box and Gwilym M. Jenkins. *Time Series Analysis: Forecasting and Control*. Holden-Day, San Francisco, CA, 1970.

[4] Danah Boyd and Kate Crawford. Critical questions for big data: Provocations for a cultural, technological, and scholarly phenomenon. *Information, Communication & Society*, 15(5):662–679, 2012.

[5] DAMA International. *The DAMA Guide to the Data Management Body of Knowledge (DAMA-DMBOK2 Revised)*. DAMA International, revised edition, 2024.

[6] Thomas H. Davenport and Jeanne G. Harris. *Competing on Analytics: The New Science of Winning*. Harvard Business Review Press, Boston, MA, updated edition, 2017.

[7] Zhamak Dehghani. *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly Media, Sebastopol, CA, 2022.

[8] Devoteam. Conversational analytics in looker: Architecture, guardrails, and implementation, 2026. Accessed: 2026-10-15.

[9] Darren Edge, Ha Trinh, Aidan Newman, Joshua Christopher, Andrew Evans, Shenghao Wang, Kevin Larson, Robert Foster, Kyle Bennett, Scott Jones, et al. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[10] Google Cloud. Best practices for configuring conversational analytics and ai agents in looker, 2024. Accessed: 2026-10-15.

[11] Google Cloud. Lookml and explore best practices for high-performance queries, 2024. Accessed: 2026-10-15.

[12] Google Cloud. Create and manage explore data agents in looker conversational analytics, 2026. Accessed: 2026-10-15.

[13] Thomas R. Gruber. A translation approach to portable ontology specifications. *Knowledge Acquisition*, 5(2):199–220, 1993.

[14] Joseph Hilger, Lulit Tesfaye, and Zachary Wahl. *Bridging Knowledge, Data, and AI: Harnessing the Semantic Layer Framework to Drive Intelligence*. Professional and Applied Computing Series. Springer Nature, Cham, Switzerland, 1st edition, 2026.

[15] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d'Amato, Gerard de Melo, Claudio Gutierrez, José Emilio Labra Gayo, Sabrina Kirrane, Sebastian Neumaier, Axel Polleres, Roberto Navigli, Axel-Cyrille Ngonga Ngomo, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4):1–37, 2021.

[16] Rob Kitchin. *The Data Revolution: Big Data, Open Data, Data Infrastructures and Their Consequences*. SAGE Publications, London, 2014.

[17] Sabina Leonelli. *Data-Centric Biology: A Philosophical Study*. University of Chicago Press, Chicago, IL, 11 2016.

[18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[19] Ikujiro Nonaka and Hirotaka Takeuchi. *The Knowledge-Creating Company: How Japanese Companies Create the Dynamics of Innovation*. Oxford University Press, New York, NY, 1995.

[20] Natalya F. Noy and Deborah L. McGuinness. Ontology development 101: A guide to creating your first ontology. Technical Report KSL-01-05, Stanford Knowledge Systems Laboratory, Stanford, CA, 2001.

[21] Dip Bharatbhai Patel. Comparative analysis of data visualization tools: Tableau, power bi & looker. *The Eastasouth Journal of Information System and Computer Science*, 1(1):55–59, 2023.

[22] Pranitha Potturi. Demystifying semantic layers in business intelligence platforms. *Sarcouncil Journal of Engineering and Computer Sciences*, 4(8):612–618, 2025.

[23] James Richardson, Kurt Schlegel, Rita Sallam, Austin Kronz, and Julian Sun. Magic quadrant for analytics and business intelligence platforms. Technical Report G00467317, Gartner, February 2021.